Prompt usados para generar el código.

1ra parte

2.

Act as an expert software developer. Please implement a Bubble Sort algorithm in Python .To perfectly match my theoretical mathematical analysis and loop invariant proofs, the implementation must strictly follow this exact structure:
1- An outer loop iterating an index i from 0 to n - 1.An inner loop iterating an index j from 0 to n - i -
2.A comparison checking if array[j] > array[j+1], followed by a swap function if true.

Requirements:
Ensure the sorting is done strictly in-place to guarantee an $O(1)$ Space complexity.

Include clear inline comments mapped to the Loop Invariant phases: Initialization (before the loops),

Maintenance (inside the inner loop), and Termination (when the array is fully sorted).Make the function clean, modular, and easy to integrate into a GUI application where I will measure execution time later."

to the test add the best cases the worst case, and add negative numbers, repeated numbers, ints and real number combinations, empty elements, one element, and add the cases that you consider appropriate for the test.

based on the first prompt make a function of quicksort algorithm with the following structure:

1- Pivot Selection (Median-of-Three): create a helper function to select the pívot, taking the first element, the medium element, and the last element of the chain of numbers.

2- Try no to create other function to make the partition of the chain numbers, use a trace callback to don't felt on over comprobation
3- make it with the same requirements of the bubble sort, and the same case of test.

3.

make a benchmark for the sort algorithms, make it with all the worst and the best cases, try to recreate all the mathematical theory of the algorithms, save the data so we can see it on a graphic, the algorithm have to be very rigorous with the time measure of the sorts clear all the errors of measuring that comes form the software and swap the first run algorithm, also make sure to measurement the operations performed by the algorithm, make a comparison between the theoretical and the experiment results.

4. having in count the benchmark cases, make a chart to compare the time and the inputs of each algorithm, in all the cases, also compare the theoretical and the results of the benchmark, to the experimental data make a linear regression of the data to erase the errors and the outliers data.

2da parte

Act as an Expert Python UI/UX Developer. Please create a Desktop GUI application using Tkinter (with ttk for modern styling) and Matplotlib (FigureCanvasTkAgg) for the Phase 2 of my Algorithms Optimization exam.

The application will serve as a dashboard to integrate, benchmark, and visualize the theoretical vs. empirical complexities of Bubble Sort and Quicksort (Median-of-Three).

the app have to contain two principal pages, the first one is for the charts and the comparison between quick sort an bubble sort, in this page the left side must contain a tool box to create different types of number strings, that could generate all the cases of the sort and a box to select the size of the numbers , the right side must have the charts, make tools to see clear the charts and to change the size of the charts, in a subpage save the data with the number of repetitions, the time, and the size.

the animation app will go in the other main page, i want a animation to the quick sort, it must have a animations of bars that the height must represent the value of the number, i want also a animation of the recursive tree and an animation of the lines of the code that are running, also what mathematical operation is being performed.

I want a bottom of pause, go back, reboot.